

Programozás Alapjai

Dr. Gergely Tamás
Dr. Kicsi András

Szegedi Tudományegyetem
Informatikai Intézet
Szoftverfejlesztés Tanszék

2025

(v1207)



Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listaszerű adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszóátlag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

A Python nyelv beépített adattípusai

Szöveg	str	Egy karaktersorozat ábrázolására való.
Szám	int, float, complex	Egész, valós és komplex számok ábrázolására való típusok.
Sorozat	list, tuple, range	Értékek sorozatának ábrázolására való típusok.
Leképezés	dict	Kulcs-érték párok ábrázolására való típus.
Halmaz	set, frozenset	Értékek halmazának ábrázolására való típusok.
Logikai	bool	Logikai értékek (True és False) ábrázolására való típus.
Bináris	bytes, bytearray, memoryview	Bináris adatok ábrázolására való típusok.
Semmi	NoneType	A None érték, azaz a semmi típusa.
		Nincs önálló karakter (char) és bájt (byte) típus!

Tartalomjegyzék

- 
- 1 Bemutatók**
 - Kurzus információk
 - Infrastruktúra szabályzatok
 - Mentorálás
 - 2 Python alapok**
 - Bevezető
 - Pénzváltás (1. verzió)
 - Pénzváltás (2. verzió)
 - 3 Alapfogalmak**
 - Alapfogalmak
 - Alapvető programelemek fogalmak
 - 4 Python feltételes ciklusok**
 - Röppálya számítás
 - Röppálya szimuláció
 - 5 Szoftverfejlesztési életciklus modellek**
 - A programozás fázisai
 - 6 Python sztringek**
 - Python sztringek
 - Basename meghatározása
 - 7 Vezérlési szerkezetek**
 - Bevezetés
 - Szekvenciális vezérlés
 - Függvények
 - 8 Python listák**
 - Python list és tuple adattípusok
 - Python listáshoz adattípusok műveletei
 - Basename és eltelt idő
 - 9 Vezérlési szerkezetek**
 - Szelekciós vezérlések
 - 10 Python iterációk**
 - Csúszólag számítása
 - 11 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 12 Python halmazok, comprehension**
 - Python set és frozenset adattípusok
 - Tér legtávolabbi pontjai
 - 13 Vezérlési szerkezetek**
 - Ismétléses vezérlések
 - 14 Python dictionary, függvény mint típus**
 - Python dict adattípus
 - Python comprehension
 - Python function adattípus
 - 15 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 16 Python fájlkezelés**
 - Fájlkezelés
 - 17 Vezérlési szerkezetek**
 - Eljárásvezérlés
 - 18 Python formátált IO**
 - Python formátált IO
 - 19 Adatkezelés, adattárolás**
 - Az adatkezelés szintjei
 - Az adattárolás jellemzői
 - 20 Python típusok**
 - Python numerikus típusok
 - 21 Absztrakt adattípusok**
 - Egyszerű adattípusok
 - Összetett adattípusok
 - 22 Python típusok**
 - 23 Absztrakt adattípusok**
 - További Python típusok
 - Python function adattípus
 - 24 Tesztelés**
 - Tesztelés
 - Tesztelés Python-ban
 - A pytest
 - 25 Vizualizáció és egyéb modulok**
 - Fájlkezelő modulok
 - Vizualizáció
 - 26 More Python**
 - A Python program végrehajtásának folyamata
 - Függvényparaméterek és argumentumok
 - Python unpacking
 - Kivételkezelés

Az int (egész) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt egész adattípus megvalósítása Python-ban.
- Értékkészlete az összes egész szám, (elméletben) nincs korlátozva.
- Az `int` *immutable* típus.



A float (*valós*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *valós* adattípus megvalósítása Python-ban.
- Értékkészlete a $[-(2 - 2^{-52}) \cdot 2^{1023}, (2 - 2^{-52}) \cdot 2^{1023}]$ zárt intervallumba eső valós számok részhalmaza úgy, hogy ebből az intervallumból minden valós szám egy adott e relatív pontossággal megközelíthető egy float adattípusbeli értékkel.
- Az értékkészlet tartalmazza még a $-\text{inf}$ és $+\text{inf}$ értékeket is.
- Ez azt jelenti, hogy (az értékkészlet határértékein belül) bármely a valós számhoz van olyan x float típusú érték, hogy

$$\left| \frac{x - a}{a} \right| \leq e.$$

- A float *immutable* típus.

A complex (*komplex*) adattípus

Python VIRTUÁLIS adattípus

- Az absztrakt *komplex* adattípus megvalósítása Python-ban.
 - A matematikai komplex számok $a + bi$ alakban (is) felírható számok, ahol $i = \sqrt{-1}$.
 - Bármely komplex szám bármely gyöke szintén komplex szám (valós számok esetében a negatív számok gyöke nem (mindig) értelmezett).
- Értékkészlete a valós típuson alapszik (felfogható egy kétdimenziós valós vektorként).
- A `complex` *immutable* típus.



Numerikus adattípusok

- Az `int`, a `float` és `complex` adattípusokat összefoglalóan numerikus adattípusoknak nevezzük.
- A numerikus adattípusok értékeinek leírására számleírást, röviden számokat használunk.
 - Az egész számokat kettes, nyolcas, tízes vagy tizenhatos számrendszerbeli leírással adhatunk meg. (Tizenhatos számrendszerben a számjegyek értéke decimálisan: A=10, B=11, C=12, D=13, E=14, F=15.)
 - Valós számok leírására tizedes törtet használhatunk, tízes exponenssel kiegészítve.
 - Komplex számok leírásakor a `j` suffixet használjuk a képzetes rész jelöléséhez.



- **Bináris számjegy:** A '0' vagy '1' karakter ([01]).
- **Oktális számjegy:** A '0' és '7' közötti számjegy karakterek ([0-7]).
- **Hexadecimális számjegy:** Egy 'a' és 'f' közötti kis vagy nagybetű, vagy egy számjegy karakter ([0-9A-Fa-f]).
- **Bináris szám:** A '0b' karakterekkel kezdődő, bináris számjegyekkel folytatódó esetleg '_' karaktert is tartalmazó karaktersorozat ('0b' [_01]+).
- **Oktális szám:** A '0o' karakterekkel kezdődő, oktális számjegyekkel folytatódó esetleg '_' karaktert is tartalmazó karaktersorozat ('0o' [_0-7]+).
- **Decimális szám:** A '0' számjegy, vagy nem '0' számjeggyel kezdődő, számjegyekkel folytatódó esetleg '_' karaktert is tartalmazó karaktersorozat ('0' | [1-9] [_0-9]*).
- **Hexadecimális szám:** A '0x' karakterekkel kezdődő, hexadecimális számjegyekkel folytatódó esetleg '_' karaktert is tartalmazó karaktersorozat ('0x' [_0-9A-Fa-f]+).
- **Valós szám:** Decimális száMLEÍRÁS után '.'-tal kezdődő decimális számból álló tizedestört-részt és/vagy 'e' karakter után előjelet és decimális számot tartalmazó exponens részt tartalmazó száMLEÍRÁS ([[:Decimális szám:]]('.'[[:Decimális szám:]]('e'[-+][[:Decimális szám:]])?|('e'[-+][[:Decimális szám:]]))).
- **Komplex szám:** Valós száMLEÍRÁS 'j' suffix-szel ([[:Valós szám:]]j).

- Egész számok

```
0
-42
+42
0x2a
-0o52
0b10_1010
```

- Valós számok

```
3.141592653589793
602_214_076e15
5.39124760e-44
2_718_281_828.459e-9
1.616_255e-35
```

- Komplex számok

```
0j
-42j
3.141592653589793j
2_718_281_828e-9j
```

- Az `nj` alakú komplex számleírás a $0 + ni$ komplex számot jelöli!

```
>>> 4+3j * 2+1j
(4+7j)
>>> (4+3j) * (2+1j)
(5+10j)
```

Numerikus típusok műveletei

Aritmetikai műveletek

- egyoperandusú műveletek
 - előjelváltás
 - `.conjugate()` konjugált (complex)
 - `.real` valós rész (complex)
 - `.imag` képzetes rész (complex)
 - `abs()` abszolútérték

```
-a  
a.conjugate()  
a.real  
a.imag  
abs(a)
```



Numerikus típusok műveletei

Aritmetikai műveletek

- kétooperandusú műveletek
 - + összeadás
 - kivonás
 - * szorzás
 - / osztás
 - // egészosztás, osztás egészrésze (`complex`)
 - % maradék (`complex`)
 - ** hatványozás

```
a + 8
7.0 - a
6.0j * 7.0j
a / 3
42 // a
a % 8.5
a ** 42.0
```

Az int (egész) adattípus műveleteinek tulajdonságai

- Az `int` adattípus műveleteire teljesülnek az aritmetika ismert azonosságai.

```
>>> 2 * (3 * 7 + 5 * 2 ** 2) == 2 ** 2 * 2 * 5 + 3 * 2 * 7
True
```

- A tárolás
 - kettes számrendszerbeli alakban (biteken),
 - az adott érték tárolásához szükséges (megvalósítás szerinti) lehető legkevesebb bájton,
 - kettes komplementes formában történik (ha n biten tároljuk az értéket, akkor a negatív $-x$ helyett $2^n - x$ tárolódik, x minden bitjét negáljuk és hozzáadunk egyet).

A float (valós) adattípus műveleteinek tulajdonságai

- A float adattípus műveleteire csak adott értéktartományban és adott pontossággal teljesülnek az aritmetika ismert azonosságai⁷.

```
>>> math.sqrt(2.0) ** 2
2.0000000000000004
>>> 3.1415927e20 + 2.718281e-20 == 3.1415927e20
True
>>> 1.7976931348623157e+308 + 1e292
inf
```

- A tárolás során külön tárolódik
 - az előjel (1 biten),
 - az értékes kettedesjegyek (az 1.bbb alakú bináris tört számjegyei, 52 biten),
 - és a kettes alapú exponens (2 hányadik hatványával szorozzuk a kettedestörtet, 11 biten).
 - A speciális értékeket (∞ , NaN) speciális bitsorozatok reprezentálják.

⁷A Python az IEEE-754 szabvány [5] „double precision” típusát valósítja meg

A complex (*komplex*) adattípus műveleteinek tulajdonságai

- A `complex` típus a `float` típus tulajdonságait örökli.
- Egy `complex` típusú érték felfogható egy `float` típusú értékpárnak.
- A `complex` érték `.real` és `.imag` része, valamint a műveletei így a `float` típus pontosságával és értéktartományával rendelkeznek.
- A `complex` *immutable* típus. Az egyes műveletek nem annak a számnak az értékét változtatják meg, amin dolgoznak, hanem egy új számot adnak vissza!

```
>>> a = b = (1 + 1j)
>>> a is b
True
>>> b += (2 - 2j)
>>> a is b
False
```

Numerikus típusok műveletei

Relációs műveletek

- kétoperandusú műveletek
 - `==` egyenlő (nem int típusnál óvatosan!)
 - `!=` nem egyenlő (nem int típusnál óvatosan!)
 - `<` kisebb (complex)
 - `>` nagyobb (complex)
 - `<=` kisebb vagy egyenlő (complex)
 - `>=` nagyobb vagy egyenlő (complex)

```
3 == a
a != 7.2j
a < 42
a > 0.2E3
a <= b
42.0 >= a
```

A relációs műveletek tulajdonságai

- A relációs műveletek esetében a Python megengedi, hogy az $a ?_1 b$ and $b ?_2 c$ alakú kifejezéseket $a ?_1 b ?_2 c$ alakban rövidítsük.

- Ez hasznos is lehet:

```
0 < x and x <= 42    # ez működik minden programozási nyelven
0 < x <= 42          # ez más nyelveken általában nem jó
```

- de túl is lehet tolni:

```
a < x and x <= y and y > b and b <= x
a < x <= y > b <= x    # ez működik, de olvashatatlan
```

- Komplex számokon nem értelmezettek a kisebb-nagyobb relációk.

Konverzió int (egész) típusra

- `int(val)`:
 - Logikai: a `False` értékből 0, a `True` értékből 1 értékű `int` lesz.
 - Valós: a törtrész elhagyásával „csonkolás” történik.
 - Sztring: tízes számrendszerbeli egészként értelmezve.
- `int(str, b)`:
 - Sztring: Az `str` sztringet `b` számrendszerbeli egészként értelmezzük.

```
>>> int(False)
0
>>> int(True)
1
>>> int(3.14)
3
>>> int(-2.71)
-2
>>> int('42')
42
>>> int('2a', 16)
42
>>> int('14', 8)
12
```

Konverzió float (*valós*) típusra

- `float(val)`:
 - Logikai: a `False` értékből 0, a `True` értékből 1 értékű float lesz.
 - Egész: a legközelebbi ábrázolható float érték lesz.
 - Sztring: a száMLEírást valósként értelmezzük.

```
>>> float(False)
0.0
>>> float(True)
1.0
>>> float(12345678901234567890)
1.2345678901234567e+19
>>> float('42')
42.0
>>> float('3.14e1000')
inf
>>> float('-2.71e-350')
-0.0
>>> float('6e23')
6e+23
```



Konverzió complex (*komplex*) adattípusra

- `complex(val)`:
 - Logikai: a `False` értékből 0, a `True` értékből 1 értékű `complex` lesz.
 - Szám: azonos (matematikai) értékű `complex` típusú érték lesz.
 - Sztring: egész, valós vagy komplex számleírásból értelmezett `complex` érték.
- `complex(re, im)`:
 - A két szám a `complex` érték valós és képzetes részeként lesz értelmezve.

```
>>> complex(False)
0j
>>> complex(True)
(1+0j)
>>> complex(12345678901234567890)
(1.2345678901234567e+19+0j)
>>> complex('42')
(42+0j)
>>> complex('2.024e3')
(2024+0j)
>>> complex('6+7j')
(6+7j)
>>> complex(3.14, 2.71)
(3.14+2.71j)
```

Matematikai függvények és konstansok

A `math` modul [6]

- Egyes matematikai függvények és konstansok, mint például a `sin`, `cos`, `log` vagy az `e` és a `pi` nem részei a nyelvnek, de egy standard Python modulban össze vannak gyűjtve.
- Használatukhoz a teljes `math` modul `import`-jára, vagy a használni kívánt konkrét elemek `math` modulból való `import`-jára lesz szükség:

```
import math

math.sin(0.0)
math.cos(math.pi)
```

```
from math import log
from math import e

log(e)
```

Matematikai függvények és konstansok

A `cmath` modul [8]

- A komplex számokat kezelő / komplex számokhoz tartozó függvények és konstansok, mint például a `sin`, `cos`, `log` vagy az `e` és a `pi` nem részei a nyelvnek, de egy standard Python modulban össze vannak gyűjtve.
- Használatukhoz a teljes `cmath` modul `import`-jára, vagy a használni kívánt konkrét elemek `cmath` modulból való `import`-jára lesz szükség:

```
import cmath
```

```
cmath.sin(cmath.pi / 2.0)  
cmath.cos(math.pi * 1j)
```

```
from cmath import log  
from cmath import e
```

```
log(e)
```

Kifejezés értéke

Prioritási előírás

- A kifejezés által jelölt értéket a kifejezés kiértékelése határozza meg, ami pedig a kifejezés felépítésén alapul.
- A kifejezés felépítéséből adódik a műveletek közötti erősségi sorrend, az úgynevezett prioritás.
- Ha egy $A \odot_1 B \odot_2 C$ alakú kifejezésben a $\odot_2$ magasabb prioritású művelet, mint $\odot_1$, akkor a kifejezés az $A \odot_1 (B \odot_2 C)$ zárójelezésnek megfelelően értékelődik ki.
- Azonos prioritású műveletek esetén Python-ban balról jobbra értékelünk ki (de más nyelvek egyes műveleteinél ez eltérhet).

```
a < b and a + - 5 * b < 7 or c != b
((a < b) and ((a + ((- 5) * b)) < 7)) or (c != b)
```

Python műveletek prioritása

műveletek	Python operátorok
zárójelezés	(...)
függvényhívás	f()
hatványozás	**
egyoperandusú prefix,	+, -, ~
multiplikatív	*, /, //, %
additív	+, -
bitléptetés	<<, >>
bitenkénti 'és'	&
bitenkénti 'kizáró vagy'	^
bitenkénti 'vagy'	
relációs, összehasonlító, tartalmazás	==, !=, <=, >=, <, >, is, is not, in, not in
logikai tagadás	not
logikai 'és'	and
logikai 'vagy'	or
feltételes	if-else

Numerikus kifejezések típusa

- Egy-egy művelet eredményének típusa az operandusainak típusától függ. Ökölszabály, hogy a művelet eredményének típusa az operandusok legbővebb típusa ($\text{Egész} \subset \text{Valós} \subset \text{Komplex}$).
- Egyes műveletek típusa lehet eleve bővebb (pl. $4 / 2 == 2.0$), mint az operandusok típusa.
- Egyes műveletek bizonyos típusokon nem értelmezettek (pl. Komplex számok között nincs kisebb-nagyobb reláció, vagy alsó-egészrész osztás és modulus ($//$, $\%$)).
- Fontos, hogy bár matematikai értelemben fennáll az $\text{Egész} \subset \text{Valós} \subset \text{Komplex}$ tartalmazás, a Python nyelv adott virtuális típusú numerikus értéke nem eleme a nyelv másik típusának ($\text{int} \not\subset \text{float} \not\subset \text{complex}$) akkor sem, ha a kettő között triviális vagy automatikus a konverzió!

```
>>> 2 == 2.0
```

```
True
```

```
>>> 2 is 2.0
```

```
False
```

Háromszög osztályozása, toleranciával

Problémafelvetés és Megoldás

- Problémafelvetés

- A lakatosműhely megkapta a háromszögek osztályozását végző programot. A használat során azt vették észre, hogy a derékszögűség felismerése nem minden esetben sikeres, egyenlőszárú derékszögű háromszögeknél pedig soha nem az. Javítani kellene a programot.
- Azt még hozzátették, hogy a gépek metrikus adatokkal (méterben) dolgoznak és 0.1 milliméteres pontosságuk van, ennél kisebb eltéréseket nem tudnak érdemben megmunkálni.

- Megoldás

- Használjuk olyan „egyenlőség relációt”, ami a megadottnál kisebb eltérés esetén egyenlőnek tekinti a két értéket.

Háromszögek osztályozása toleranciával [1/2]

haromszog_tol.py [1-23]

```
1#!/usr/bin/env python3
2"""
3Programozás_Alapjai_előadás_példaprogram
4SZTE_TTIK, Informatikai_Intézet, Szoftverfejlesztés_Tanszék
5Gergely_Tamás, gertom@inf.u-szeged.hu, 2024-05-31
6cc-by-nc-nd
7
8Milyen_háromszöget_határoz_meg_három_pozitív_valós_szám,
9mint_a_háromszög_három_oldalhosszúsága?_A_valós_érték
10pontatlansága_miat_toleranciával_számlolunk.
11"""
12def equals(x,y):
13    return abs(x-y) <= 1e-4
14
15def main():
16    print("Kérem_a_háromszög_oldalhosszúságait!")
17    a = float(input("a?_"))
18    b = float(input("b?_"))
19    c = float(input("c?_"))
20    if a < b:
21        a, b = b, a
22    if a < c:
23        a, c = c, a
```

Háromszögek osztályozása toleranciával [2/2]

haromszog_tol.py [25–42]

```
25     if b <= 0 or c <= 0:
26         print("Nem_háromszög!")
27     elif a >= b + c:
28         print("Nem_háromszög!")
29     elif equals(a, b) and equals(b, c) and equals(a, c):
30         print("Szabályos_háromszög.")
31     elif equals(a, b) or equals(b, c) or equals(a, c):
32         if equals(a ** 2, b ** 2 + c ** 2):
33             print("Egyenlőszárú_derékszögű_háromszög.")
34         else:
35             print("Egyenlőszárú_háromszög.")
36     elif equals(a ** 2, b ** 2 + c ** 2):
37         print("Derékszögű_háromszög.")
38     else:
39         print("Egyéb_háromszög.")
40
41 if __name__ == "__main__":
42     main()
```

Szinusz(x) kiszámítása

Pontosság

- A $\sin(x)$ -et kiszámító algoritmusnál beszéltünk arról, hogy milyen előnyei vannak, ha az x értékét 0 közelébe transzformáljuk.
- Nézzük meg, mit okoz a `float` típus pontatlansága a transzformáció illetve annak hiánya esetében.



Szinusz [1/2]

szinusz_prec.py [1-19]

```
1#!/usr/bin/env python3
2"""
3Programozás Alapjai előadás példaprogram
4SZTE TTK, Informatikai Intézet, Szoftverfejlesztés Tanszék
5Gergely Tamás, gertom@inf.u-szeged.hu, 2024-06-04
6cc-by-nc-nd
7
8sin(x) közelítő értékének kiszámítása a sin(x) függvény alkalmazása nélkül.
9x értékét transzformáljuk a [0, 2*Pi) intervallumba.
10"""
11from math import pi
12from sys import argv
13from sys import stderr
14from termcolor import colored
15
16EPSZ = 1e-10
17
18def transform(x):
19    return x % (2.0 * pi)
```

Szinusz [2/2]

szinusz_prec.py [21–47]

```
21 def szinusz(x, debug=False):
22     global EPSZ
23     osszeg = 0.0
24     j = 2
25
26     tag = x
27     xx = x ** 2
28
29     while True:
30         osszeg += tag
31         if debug:
32             print(colored(f"[LOG] tag={tag:30.10f}, osszeg={osszeg:30.10f}", 'yellow'), file=stderr)
33         tag = -(tag * xx / j / (j + 1))
34         j += 2
35         if abs(tag) < EPSZ:
36             break
37     return osszeg
38
39 def main():
40     dbg = len(argv) > 1
41     x = float(input("Kérem sin(x)-hez az x-értékét: "))
42     print("sin(%.5f)=%.10f" % (x, szinusz(x, dbg)))
43     xt = transform(x)
44     print("sin(%.5f)=%.10f" % (x, szinusz(xt)))
45
46 if __name__ == "__main__":
47     main()
```